西北工业大学实验报告

课程名称:高级语言程序设计实验

实验课次周次:第三次（第十一周）

实验地点:实验大楼 B107

学生姓名：张洛宾

学　　号:2024302696

实验时间:2026-05-16

提交时间:2025-05-22

成绩评定：（满分为 100 分） 指导教师评语： 指导教师签名：

目　录

1　UML 实验三 UML 类图 2

1.1　实验内容（要求、算法、步骤和方法） 2

1.2　UML 结果文件说明 2

1.3　UML 实验结果 2

2　实验四 对象图 6

2.1　实验内容（要求、算法、步骤和方法） 6

2.2　UML 结果文件说明 6

2.3　UML 实验结果 6

3　UML 实验五 时序图与协作图 9

3.1　实验内容（要求、算法、步骤和方法） 9

3.2　UML 结果文件说明 10

3.3　UML 实验结果 10

4　UML 实验六 状态图 13

4.1　实验内容（要求、算法、步骤和方法） 13

4.2　UML 结果文件说明 13

4.3　UML 实验结果 13

5　实验总结 16

5.1　实验中遇到的问题以及解决方法 16

5.2　实验体会 16

1 UML 实验三 UML 类图

1.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

1.2 UML 结果文件说明

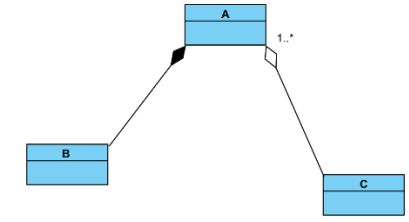
UML03.vpp--- 实验三的所有 UML 图

1.3 UML 实验结果

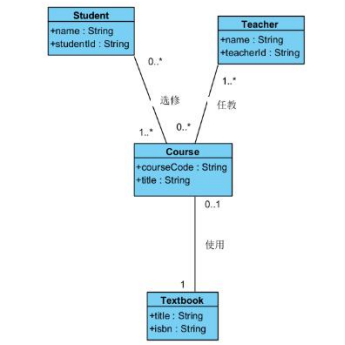
1. 类是面向对象编程中的基本概念，用于描述具有相同属性和行为的对象的抽象模板。它定义了对象的数据（属性）和操作（方法）。

   类的三要素:属性,方法,关系

2. Public 用+ private 用- Protected 用# Default 用~(或无符号)

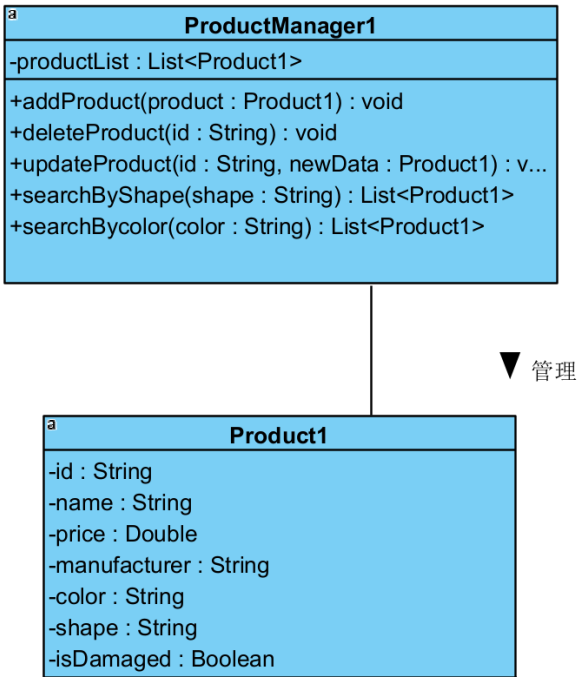
3. 

   按照要求的实验三,C 中有表示数字的标志,然后 B 只有单独一个

4. 

按照问题 4 的要求,学生和老师都有自己的属性 name 和 id,其中学生和教师分别与 course 类产生关联,其中一个学生选修多个课程,也可以不选,所以在关系中使用 1..*和 0..*来表示.使用选修和任教来表示,同时课程的关系与教材的关系同理.

5.

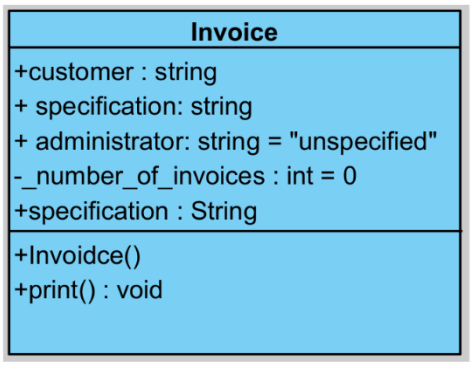


根据要求来产生两个类,Product 类用于构建产品将所有属性添加到这个类.然后使用 ProductManager 类来管理这个 Product 类,同时将给定的方法添加其中.

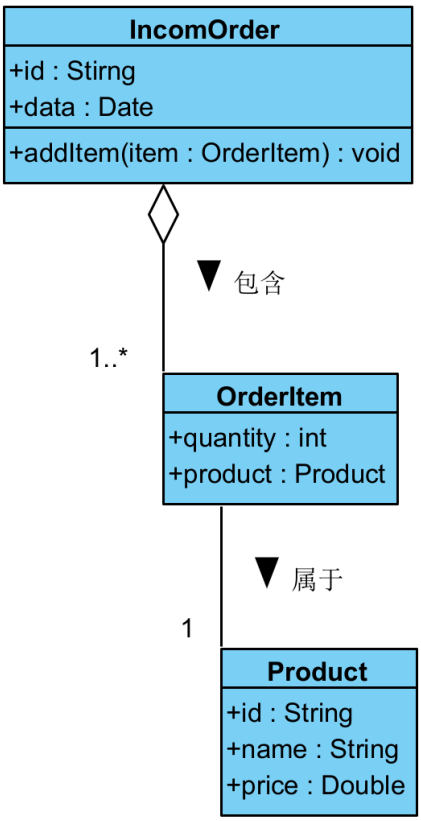
第六题也在这个图当中,添加了所需的属性和方法即可.

6. 如上图

7.



按照代码描述类即可.



8.

首先，IncomeOrder（入库单）与 Product（产品信息）之间的双向 1 对 1 关联是不合理的，因为一个入库单通常包含多个产品（通过 OrderItem 关联），而一个产品也可能出现在多个入库单中，因此 IncomeOrder 不应直接与 Product 关联，而应通过 OrderItem 间接关联。

其次，OrderItem 与 IncomeOrder 之间的空心三角形箭头（通常表示泛化/继承关系）是错误的，因为 OrderItem 是入库单的组成部分，应使用组合关系（实心菱形箭头）表示 IncomeOrder 由多个 OrderItem 组成。正确的模型应为：IncomeOrder 与 OrderItem 之间是 1 对多的组合关系（实线+实心菱形箭头），OrderItem 与 Product 之间是多对 1 的关联（单向箭头），表示每个 OrderItem 对应一个具体的 Product，而 Product 不直接反向关联 OrderItem。这样调整后，模型更符合实际业务逻辑，即入库单由多个订单项组成，每个订单项引用一个产品信息。

分析与讨论

1. 类图在软件建模过程中扮演着核心角色，它通过可视化的方式清晰展现系统的静态结构，帮助开发团队理解业务逻辑和设计架构。首先，类图能够直观描述系统中各个类及其相互关系，比如属性、方法以及类之间的关联、继承或组合等，这种结构化表达让复杂系统的设计思路一目了然。其次，

它作为开发人员与业务人员之间的沟通桥梁，用标准化的 UML 符号消除自然语言可能带来的歧义，确保需求理解的准确性。此外，类图还能为后续的代码实现提供蓝图，程序员可以直接根据类图定义类结构、编写属性和方法，减少设计到实现的转换误差。在系统维护阶段，类图作为文档的一部分，能快速帮助新成员理解现有代码结构，或者辅助分析修改影响范围。总的来说，类图贯穿软件生命周期的各个阶段，从需求分析到系统设计再到代码实现和维护，都是不可或缺的工具。

2. 首先，画类图就像搭积木，每个类最好只做一件事（比如学生类管学生信息，成绩类管成绩），别把所有功能都塞进一个类里，不然代码会很难改。其次，要分清类之间的关系，比如"学生有成绩"用实线箭头表示，"大学生继承学生"用空心三角箭头.另外，属性和方法需要标清楚+（公开）、-（私有）这些符号。最重要的是类图要和 Java 代码对应上，比如类图画了 Student 类，写代码时就要有这个类，这样才不会出错。

2 实验四 对象图

2.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

2.2 UML 结果文件说明

具体代码请参见附件中电子版的文件。结果文件的简要说明如下：

UML04.vpp---包含实验四的所有 UML 文件.

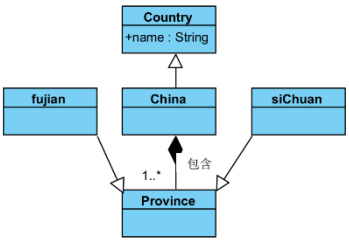
2.3 UML 实验结果

（包括绘制的 UML 图，UML 图的说明，以及实验要求中需要书面回答的内容）

1. 对象是面向对象编程（OOP）中的核心概念，指在程序中具有明确身份和行为的实体。它是由类（Class）创建的实例，包含了具体的数据（属性）和可执行的操作（方法）。

   三大特征：封装,继承,多态.

2. 矩形框表示,匿名对象表示,关联关系表示

3. 

   对象的名称是 China

   图的意思是,China 是一个 Country 的实例,包含两个省份实例 fuJian 和 siChuan,这两个对象都是类 Province 实例.并且通过组合关系表示中国包含这两个省份.

   类图如上

4. 对象图主要用于展示系统在某个特定时刻的具体对象实例及其关系,特别适合以下场景：比如调试代码时需要查看对象间的实际关联，或者向团队

成员解释复杂对象之间的交互时。它的最大优点是能直观呈现对象实例的属性和关系，让人一眼就能看懂系统在运行时的状态，这对理解业务逻辑特别有帮助。不过它也有明显缺点，主要是只能反映静态状态而无法展示动态行为，比如方法调用过程。另外当系统规模较大时，绘制和维护对象图会比较麻烦，容易出现杂乱不清的情况。总的来说，对象图适合用来辅助理解关键业务对象的关系，但不适合用来描述整个系统的完整行为。

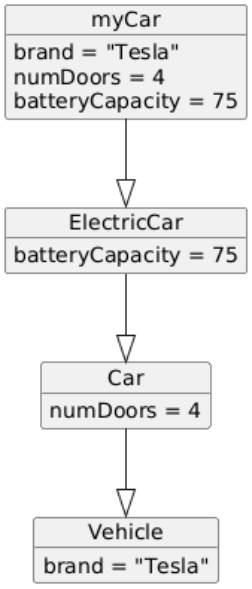
分析与讨论

1. 类和对象的关系可以简单理解为"模板"和"实例"的关系。类定义了对象的共同特征和行为，而对象是类的具体实现。

   a) 类：Student（学生类）
   对象：张三（学号 2023001）、李四（学号 2023002）
   b) 类：Car（汽车类）
   对象：我的红色大众（车牌京 A12345）、邻居的白色丰田（车牌京 B67890）

2. 类作为抽象模板，定义对象的共性特征（属性和方法），属于概念层面。对象则是类的具象实例，具有确定的状态（属性值）和行为表现。

   类体现系统静态结构，关注"是什么"（is-a）和"有什么"（has）。对象展现运行时动态交互，关注"怎么做"（do）和"状态如何"（state）。

3. 

   1. myCar 是 ElectricCar 类的一个实例对象

2. 继承关系：ElectricCar → Car → Vehicle
3. 每个对象的属性值
4. 代码如下：

```
// 父类：Vehicle
class Vehicle {
    private String brand;

    public Vehicle(String brand) {
        this.brand = brand;
    }

    public void start() {
        System.out.println("Vehicle is starting...");
    }
}

// 子类：Car
class Car extends Vehicle {
    private int numDoors;

    public Car(String brand, int numDoors) {
        super(brand);
        this.numDoors = numDoors;
    }

    public void honk() {
        System.out.println("Car is honking!");
    }
}

// 子类：ElectricCar
class ElectricCar extends Car {
    private int batteryCapacity;

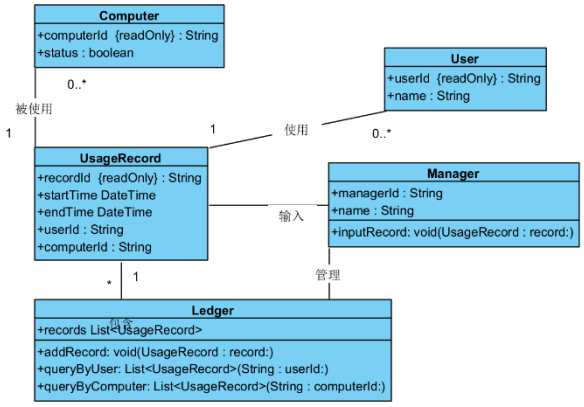
    public ElectricCar(String brand, int numDoors, int batteryCapacity) {
        super(brand, numDoors);
        this.batteryCapacity = batteryCapacity;
    }

    public void charge() {
        System.out.println("Electric car is charging...");
    }
}

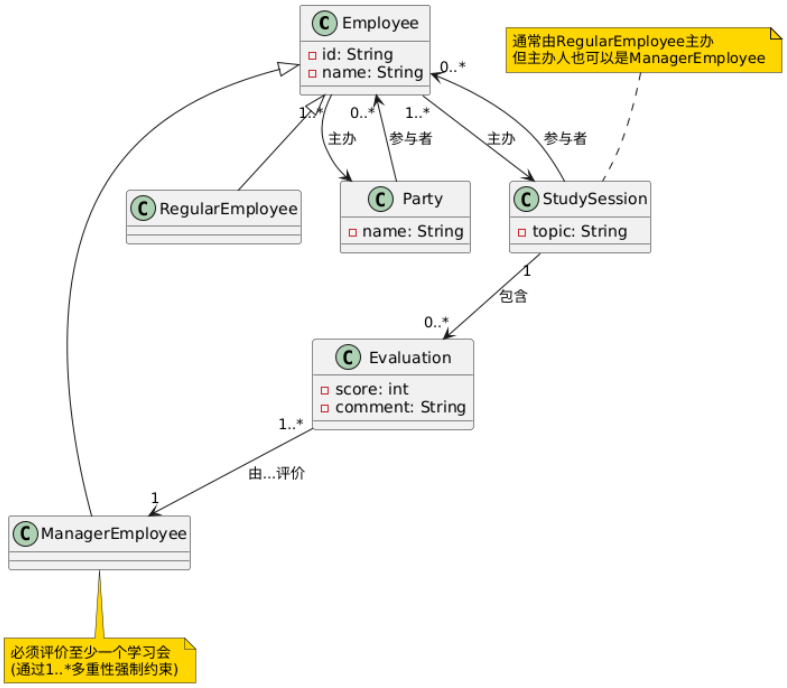
// 主类
public class Main {
    public static void main(String[] args) {
        ElectricCar myCar = new ElectricCar("Tesla", 4, 75);
        myCar.start();
        myCar.honk();
        myCar.charge();
    }
}
```

实验内容

1.



2.



3 UML 实验五 时序图与协作图

3.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

3.2 UML 结果文件说明

具体代码请参见附件中电子版的文件。结果文件的简要说明如下：

UML05--- 包含实验五的所有 UML 图

3.3 UML 实验结果

1. 区别:

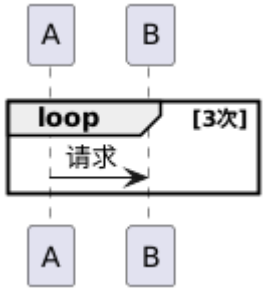
   a) 侧重点不同,时序图强调**时间顺序**，按消息调用顺序展示交互流程，适合分析时间敏感场景。协作图强调**对象结构关系**,分析对象间的静态连接

   b) 表现形式不同,时序图用垂直时间轴+生命线,协作图用对象节点+连接线

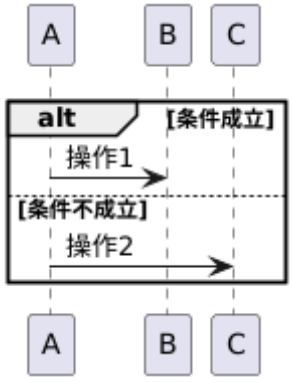
   联系:

   a) 同样是交互图,可以相互转换,结合使用可以全面理解系统行为.

2. 顺序图中使用 loop 组合片段,在片段内标注循环条件



判断使用 alt 和 else 组合片段,表示分支逻辑



协作图中 Loop 在消息前标注*[循环条件],在消息编号后加[条件],不同分支使用不同消息编号.

3. 在 UML 交互图中，**消息**是对象之间传递的通信信号，用于描述一个对象对另一个对象的请求或触发行为，它可以代表方法调用、信号发送或异步通信等交互形式。消息在交互中扮演着**行为驱动者**的角色，通过定义对象之间的动态协作关系来推动系统流程的执行。不同类型的消息（如同步、异步、创建或销毁消息）能够精确刻画系统运行时对象间的控制流和数据流，从而帮助开发者理解系统的动态行为逻辑。消息的传递顺序和条件决定了交互的完整过程，是交互图（如顺序图、协作图）的核心建模元素。
4. 交互图消息分为同步消息,异步消息,返回消息,自调用消息. 步消息（实心箭头→）表示发送者等待接收者处理完成后再继续执行，常用于普通方法调用；异步消息（开放箭头→）则无需等待返回即可继续执行，适用于事件触发或非阻塞通信；返回消息（虚线箭头--→）用于表示方法调用的响应或结果返回；自调用消息（箭头指向自身）描述对象内部方法的递归或嵌套调用。

   6- **初始状态**：test = 1

   循环执行 3 次（i = 1, 2, 3）：

   第 1 次循环（i=1）：

   fact(test)：fact(1) = 1! = 1

   test = test + fact(test) = 1 + 1 = 2

   第 2 次循环（i=2）：

   fact(test)：fact(2) = 2! = 2

   test = test + fact(test) = 2 + 2 = 4

   第 3 次循环（i=3）：

   fact(test)：fact(4) = 4! = 24

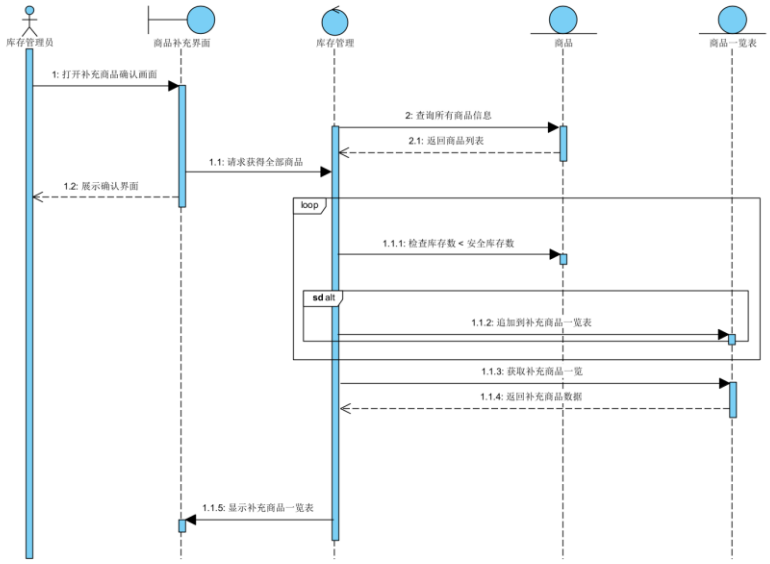
   test = test + fact(test) = 4 + 24 = 28

   最终打印：print(test) 输出 28

分析与讨论

在软件系统建模过程中，交互图（如顺序图和协作图）发挥着至关重要的作用，它们通过可视化对象之间的动态交互行为，帮助开发团队准确理解和设计系统的运行时逻辑。交互图能够清晰地展现消息传递的顺序、条件分支和循环结构，使复杂的控制流变得直观可读，从而有效减少需求理解偏差和设计缺陷。同时，它们作为沟通桥梁，让技术人员与非技术人员都能快速把握系统关键流程，促进团队协作。在系统分析和架构设计阶段，交互图不仅能验证业务逻辑的正确性，还能为后续的代码实现提供明确指导，是确保软件质量与可维护性的重要工具。

实验内容



该顺序图完整展现了库存补充确认的业务交互过程，从库存管理人发起操作开始，系统通过边界类的商品补充界面接收指令，由控制类的库存管理模块协调处理核心逻辑，包括获取全部商品数据、逐项检查库存数与安全库存数的关系，并将需要补充的商品动态添加到一览表中，最终通过界面层返回可视化结果。

4 UML 实验六 状态图

4.1 实验内容（要求、算法、步骤和方法）

请参见实验指导书。

4.2 UML 结果文件说明

具体代码请参见附件中电子版的文件。结果文件的简要说明如下：

UML06---包含实验 6 的所有图

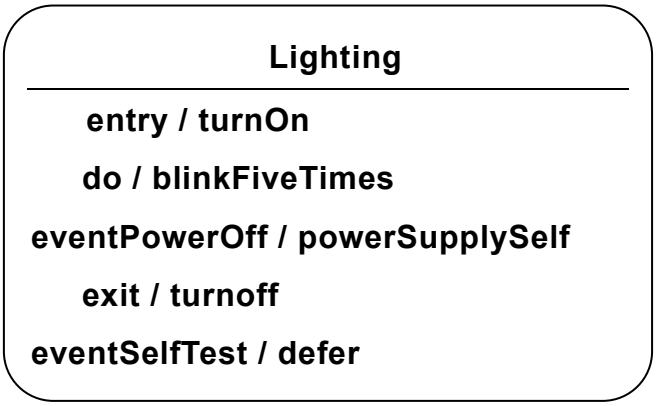
4.3 UML 实验结果

1. 在 UML 中，状态表示对象在其生命周期中满足某些条件、执行某些活动或等待某些事件时的一种状况，它由五个核心部分组成：名称（状态的标识）、进入/退出动作（进入或离开状态时执行的操作）、内部转换（不引起状态改变的内部事件处理）、子状态（嵌套的细化状态）以及延迟事件（暂不处理而排队的事件）。
2. 在 UML 中，转换表示对象从一个状态到另一个状态的变化过程，它由五个关键要素组成：源状态（转换开始时的初始状态）、目标状态（转换完成后的到达状态）、触发事件（引起转换发生的信号或条件）、监护条件（布尔表达式，决定是否允许转换）以及效果（转换过程中执行的动作或行为）。
3. 在 UML 中，触发事件是指引发状态转换的特定信号或条件，它是对象从一种状态转移到另一种状态的直接诱因。根据事件的性质和来源，可以具体分为四类典型事件：调用事件（对象接收到方法调用请求，如 onClick()）、信号事件（异步通信中显式触发的命名信号，如 emergencyAlert）、变化事件（基于布尔表达式满足的条件触发，如 when(库存量<阈值)）、以及时间事件（由绝对时间或相对时间条件引发，如 after(2 秒)或 at(2023-12-31)）。
4. 在 UML 中，改变事件和监护条件虽然都用于控制状态转换的触发时机，但存在本质差异。改变事件（如 when(温度>100℃)）是持续性条件监测，它会主动触发转换（当条件从假变为真时立即执行），属于事件驱动机制；而监护条件（如[库存量<安全值]）是瞬时性布尔判断，仅在转换被触发事件激活后才会评估，决定是否允许转换继续，属于条件过滤机制。两

者关键区别在于：改变事件本身作为转换的触发源，会持续监听系统状态变化；监护条件则依附于已有触发事件，仅作为附加的验证关卡。

5. 在 UML 中，转换（Transition）根据其触发机制和行为特征可分为四种核心类型：外部转换（External Transition）是最常见的转换形式，由事件触发并导致源状态退出和目标状态进入，伴随完整的进入/退出动作执行；内部转换（Internal Transition）不会引起状态改变，仅在当前状态内响应事件并执行指定动作，避免不必要的状态进出开销；自转换（Self-Transition）是特殊的外部转换，其源状态与目标状态相同，会触发当前状态的退出和重新进入流程；完成转换（Completion Transition）则无需显式事件触发，当状态内部活动自动执行完毕后立即发生，常用于建模连续自动化流程。
6. 在面向对象系统建模中，状态描述对象在其生命周期中的特定状况，而对象是具有明确身份和行为的实体实例。

   a） 支票已付,汽车停在那里,小王正在工作都是状态.

7. 

状态名是Lighting,进入动作是 entry/turnOn,进入状态的时候自动开启电源,内部动作是 do/blinkFiveTimes,状态持续执行闪烁 5 次.退出动作是 exit/turnOff,离开状态自动执行关闭电源.eventPowerOff 是,触发动作执行自供电逻辑,eventSelfTest,标记为 defer 暂不处理,事件进入延迟队列.

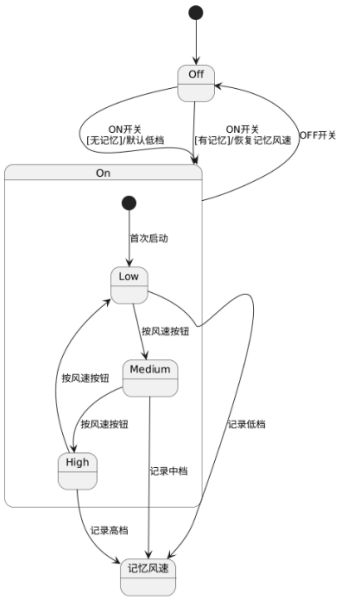
8. 描述了一个烧水开关系统的行为逻辑，系统包含"Off"和"On"两个主要状态。当处于关闭（Off）状态时，系统接收"turnOn"开启指令后会根据当前水位条件进行分支判断：若检测到"没水"则保持关闭状态不做任何操作；若检测到"有水"则执行"烧水"动作并切换到开启（On）状态。在开启状态下，系统可能遇到两种转换情况：一种是正常情况下的"水开了"事件，此时系统执行"turnOff"关闭操作返回初始状态；另一种是异常情况下的"烧坏"事件，这将直接导致系统终止运行。

9. 该状态图描述了一个课程评价系统的流程，从学生注册课程开始，经过实验 1 和实验 2 的完成（蓝色部分表示实验模块的状态转换，包括退出实验的条件 exit/lab=true），随后进入团队项目阶段，项目完成后进入考试状态。考试结果分为及格和不及格，及格则课程通过，不及格则课程不通过并进入补考状态。蓝色部分强调了实验环节的完成条件和状态切换，是课程评价流程中的关键节点，直接影响后续考试和课程最终结果的判定。

分析与讨论

1. 状态图在 UML 建模中扮演着动态行为建模的关键角色，它通过可视化对象在其生命周期中的状态变迁，将复杂的业务逻辑转化为直观的图形化表达。这种建模方式使系统行为既符合物理设备的操作规范，又能覆盖正常流程和异常处理场景，为开发人员提供了精准的状态转换蓝图，有效避免了因业务理解偏差导致的实现错误，同时这种图形化表达也大大提升了团队沟通效率，确保技术实现与领域需求保持高度一致。
2. 在系统行为具有明确状态划分且状态转换受特定事件或条件触发的场景下，特别适合引入状态图进行建模。当业务对象需要描述其生命周期中不同阶段的行为差异（如订单从"待支付"到"已发货"的流程），或者系统需要对异常状态（如设备故障、交易超时）进行专门处理时，状态图能通过状态节点、转换箭头和监护条件，精确表达对象在各种情况下的响应规则。

实验内容



这张电风扇状态图清晰地展现了设备运行的完整生命周期，通过"Off"和"On"两个核心状态构建了基本框架。当设备处于"Off"关闭状态时，触发 ON 开关会产生两种转换路径：若系统没有记忆风速则默认以低档启动，形成一条指

向"On"状态内低档子状态的转换；若存在记忆风速则直接恢复上次档位，这条转换巧妙地绕过了默认路径。在"On"状态的内部，低、中、高三档风速通过环形箭头相互连接，形成一个闭合的档位切换循环，每次按下风速按钮都会沿着这个循环弧线平滑过渡到下一档位。

5 实验总结

5.1 实验中遇到的问题以及解决方法

问题 1 绘制 UML 类图的时候,没有找到 static 属性的设置方法
解决 1 经过搜索,发现需要在属性前面添加"_"即可表示 static 修饰

问题 2 绘制 UML 类图的时候,对于判断类之间的关系不是很熟练,判断类之间的关联箭头比较困难
解决 2 通过复习 UML 类图的课本知识,以及类图之间关系表示,以及关联箭头的设置项,比如多重性等问题.

问题 3 绘制 UML 对象图的过程中,对于类图和对象图还是有些混淆
解决 3 通过对比复习 UML 类图和对象图之间的联系与区别,类图实际上是对象图的蓝图,对象图是类图的实例.

问题 4 在时序图当中,循环和条件的 UML 表示不会画,
解决 4 经过搜索,发现 visual paradigm 的 loop 和 alt 是在循环复合判断,然后点开规格说明选择操作种类,来画 loop 和 alt.然后在交互操作元当中选择约束来说明 loop 和 alt 的条件.

5.2 实验体会

通过本系列 UML 实验，我掌握了类图、对象图、时序图与状态图的核心建模方法，深入理解了面向对象设计中类与对象的关系、动态交互逻辑以及状态转换机制。使用 Visual Paradigm 等工具绘制标准 UML 图的实践，增强了我的系统抽象能力和建模规范性。对组合/聚合关系、消息传递类型、状态转换触发条件等概念的辨析，让我认识到精准建模对软件设计的重要性。未来建

议增加真实业务场景的复杂案例建模训练，并引入团队协作建模任务，以提升解决实际工程问题的能力。